

算法设计与分析期中考试试卷

Amadeus

第一题 (5 分, 单选题)

对长度为 n 的数组执行堆排序, 其最坏情况时间复杂度的紧的渐进界为

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n\sqrt{n})$
- D. $\Theta(n^2)$
- E. $\Theta(n^2 \log n)$

第二题 (5 分, 单选题)

主定理 (Master Theorem): 设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 由递归式 $T(n) = aT(n/b) + f(n)$ 定义。则 $T(n)$ 有如下渐进界:

1. 若存在常数 $\epsilon > 0$ 使得 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。
3. 若存在常数 $\epsilon > 0$ 使得 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 且存在常数 $c < 1$ 及充分大的 n 使得 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$ 。

根据主定理, 递归式 $T(n) = 16T(\frac{n}{4}) + 10^{2026} \cdot n$ 的解为

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(n^2 \log n)$
- E. $\Theta(n^4)$

第三题（5 分，单选题）

以下排序算法中，最坏情况下额外空间复杂度最低（即 $O(1)$ ）的是：

- A. 归并排序
- B. 计数排序
- C. 快速排序
- D. 堆排序
- E. 基数排序

第四题（5 分，单选题）

将 n 个球随机独立均匀地投入 n 个盒子中，最大负载的高概率渐进界为

- A. $\Theta(1)$
- B. $\Theta(\log n)$
- C. $\Theta\left(\frac{\log n}{\log \log n}\right)$
- D. $\Theta(\sqrt{n})$
- E. $\Theta(n)$

第五题（5 分，多选题，可能有 1-5 个正确选项，完全正确才得分）

以下说法中，正确的有哪几个？

- A. 任何基于比较的排序算法在最坏情况下至少需要 $\Omega(n \log n)$ 次比较
- B. 随机化快速排序在最坏情况下的时间复杂度为 $O(n \log n)$
- C. 计数排序是一种稳定的排序算法，其时间复杂度为 $O(n + k)$ ，其中 k 为元素取值范围
- D. 在简单均匀哈希假设下，使用链表法的哈希表中，假设每个已存关键字等概率被查询，一次成功查找的期望时间为 $\Theta(1 + \alpha)$
- E. 基数排序的时间复杂度与待排序元素的位数无关

第六题 (10 分)

给定 k 个已排序（升序）的链表，总共包含 n 个元素。

例如：lists = [[1, 4, 5], [1, 3, 4], [2, 6]]，合并结果为 [1, 1, 2, 3, 4, 4, 5, 6]。

- (1) (5 分) 设计一个 $O(n \log k)$ 时间的算法，将它们合并为一个有序链表。要求：描述算法思想，说明使用的数据结构，并写出算法的关键步骤。
- (2) (5 分) 证明算法的正确性，并分析时间复杂度。要求写出时间复杂度的推导过程，说明为何总时间为 $O(n \log k)$ 。

第七题 (10 分)

给定一个长度为 n 的整数数组 $A[1..n]$ ，逆序对定义为满足 $i < j$ 但 $A[i] > A[j]$ 的数对 (i, j) 。

- (1) (5 分) 设计一个基于归并排序的分治算法，在 $O(n \log n)$ 时间内计算数组中逆序对的总数。要求：描述算法的分解、合并过程，特别说明在归并（Merge）步骤中如何统计跨越两个子数组的逆序对数目，并写出伪代码或详细的算法描述。
- (2) (5 分) 证明算法的正确性（即算法确实计算了所有逆序对），并分析时间复杂度。要求写出递归式并求解。

第八题 (10 分)

考虑求递归式 $T(n) = 3T(\frac{n}{3}) + n\sqrt{\log n}$ 的紧的渐进上界。

- (1) (3 分) 尝试用主方法求解该递归式。若主方法不能直接应用，请说明原因。
- (2) (4 分) 利用递归树法计算该递归式的渐进上界，要求画出递归树结构，标明每层的代价，并给出求和过程。
- (3) (3 分) 用代换法证明你在（2）中得到的渐进上界成立。要求写出猜测、代入步骤和归纳证明的完整过程。

第九题 (10 分)

课上的 Randomize-In-Place 算法代码如下：

```
RANDOMIZE-IN-PLACE(A, n)
    for(i = 1; i <= n; i++)
        swap(A[i], A[RANDOM(i, n)])
```

如果改为以下版本，每次独立地随机选择两个位置进行交换：

```
for(i = 1; i <= n; i++)
    swap(A[RANDOM(1, n)], A[RANDOM(1, n)])
```

即每次迭代中，独立地从 $\{1, \dots, n\}$ 中随机选取两个位置（可以相同）并交换对应元素。该算法是否能产生均匀随机排列？给出证明或反例。

第十题 （15 分）

考虑 n 个元素的线性时间选择算法（SELECT，又称中位数的中位数算法）。该算法的核心思想是：将数组分成 $\lceil n/5 \rceil$ 组，每组 5 个元素（最后一组可能不足 5 个），找每组的中位数，然后递归地找所有中位数的中位数作为 pivot，以此保证划分的平衡性。

- (1) (5 分) 详细描述该算法的完整步骤，并写出伪代码。
- (2) (5 分) 写出该算法时间复杂度的递归式。
- (3) (5 分) 如果将每组的大小从 5 改为 3（即将数组分成 $\lceil n/3 \rceil$ 组，每组 3 个元素），该算法是否仍能保证最坏情况下 $O(n)$ 的时间复杂度？请写出此时的递归式并给出完整的分析过程，说明结论及原因。

第十一题 （10 分）

定义哈希函数 $h(x) = \lfloor m(ax - \lfloor ax \rfloor) \rfloor$ ，其中 m 为哈希表大小， a 从 $(0, 1)$ 上均匀随机选取（注意 $ax - \lfloor ax \rfloor$ 表示取 ax 的小数部分）。

- (1) (3 分) 写出全域哈希族（Universal Hash Family）的定义。
- (2) (5 分) 对于任意两个不同的正整数 $x_1 \neq x_2$ ，证明：当 a 从 $(0, 1)$ 上均匀随机选取时，碰撞概率满足 $\Pr[h(x_1) = h(x_2)] \leq \frac{2}{m}$ 。
- (3) (2 分) 如果使用该哈希方案实现链表哈希，一次不成功查找的期望时间上界会变为多少？（用装载因子 $\alpha = n/m$ 表示。）

第十二题 （10 分）

对长度为 n 的数组执行随机化快速排序。设排序后第 i 小的元素为 z_i 。定义指示器随机变量 $X_{ij} = I\{z_i \text{ 与 } z_j \text{ 在排序过程中被直接比较}\}$ ，其中 $1 \leq i < j \leq n$ 。

- (1) (6 分) 求 $\Pr[X_{ij} = 1]$ 的值，并利用该结果证明随机化快速排序的期望比较次数为 $O(n \log n)$ 。要求：给出概率的推导过程，以及期望比较次数的完整求和与放缩。
- (2) (4 分) 设总比较次数为 $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$ ，证明 $\text{Var}[X] = O(n^2)$ ，说明比较次数集中在期望值附近。（提示：利用指示器随机变量两两之间的协方差进行分析。）

参考答案

一. (5 分, 单选题) 答案: B

堆排序的最坏情况时间复杂度为 $\Theta(n \log n)$ 。堆排序建堆需要 $O(n)$ 时间, 每次删除堆顶并调整需要 $O(\log n)$ 时间, 共 n 次删除, 故总时间为 $\Theta(n \log n)$, 且该界是紧的。

二. (5 分, 单选题) 答案: C

递归式 $T(n) = 16T(\frac{n}{4}) + 10^{2026} \cdot n$ 。

此处 $a = 16$, $b = 4$, $\log_b a = \log_4 16 = 2$ 。 $f(n) = 10^{2026} \cdot n = \Theta(n)$ 。

取 $\epsilon = 1 > 0$, 则 $f(n) = \Theta(n) = O(n^{\log_b a - \epsilon}) = O(n^{2-1}) = O(n)$, 满足主定理情形 1。

因此 $T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$ 。

三. (5 分, 单选题) 答案: D

堆排序的额外空间复杂度为 $O(1)$ (原地排序)。

- 归并排序需要 $O(n)$ 额外空间用于合并。
- 计数排序需要 $O(k)$ 额外空间 (k 为取值范围)。
- 快速排序递归调用栈需要 $O(\log n)$ (平均) 或 $O(n)$ (最坏) 额外空间。
- 基数排序需要 $O(n + k)$ 额外空间。

四. (5 分, 单选题) 答案: C

将 n 个球随机独立均匀地投入 n 个盒子, 最大负载以高概率为 $\Theta(\frac{\log n}{\log \log n})$ 。这是经典 balls-into-bins 问题的标准结论: 当 $m = n$ 时, 最大负载以高概率为 $\Theta(\frac{\log n}{\log \log n})$, 而非 $\Theta(\log n)$ 。

五. (5 分, 多选题) 答案: A、C、D

- **A 正确**。基于比较的排序算法的决策树下界为 $\Omega(n \log n)$ 。
- **B 错误**。随机化快速排序的期望时间复杂度为 $O(n \log n)$, 但最坏情况仍为 $O(n^2)$ (例如, 极端不走运的随机选择仍可能每次都选到最值元素)。
- **C 正确**。计数排序是稳定的, 时间复杂度 $O(n + k)$ 。
- **D 正确**。在简单均匀哈希假设下, 成功查找的期望时间为 $\Theta(1 + \alpha)$, 其中 $\alpha = n/m$ 为装载因子。
- **E 错误**。基数排序的时间复杂度为 $O(d(n + k))$, 其中 d 为位数, 与位数有关。

六. (10 分) 合并 k 个有序链表

(1) (5 分) 算法设计

算法思想: 使用最小堆 (Min-Heap) 维护当前 k 个链表的头元素。每次从堆中取出

最小元素加入结果链表，然后将该元素所在链表的下一个元素（若存在）插入堆中。重复直至堆空。

数据结构：最小堆（优先队列），堆中每个元素为结点引用，比较关键字为结点值。

算法步骤：

1. 创建空的最小堆 H 。
2. 遍历 k 个链表，若链表非空，将其头结点插入 H 。
3. 当 H 非空时：
 - 从 H 中取出最小结点 $node$ ，追加到结果链表尾部。
 - 若 $node.next \neq \text{null}$ ，将 $node.next$ 插入 H 。
4. 返回结果链表。

(2) (5 分) 正确性证明与时间复杂度分析

正确性：循环不变式——堆中始终包含各链表当前未合并的最小元素候选。每次从堆中取出的元素是当前所有未合并元素中的最小值，因此结果链表保持升序。由于每个元素都被插入堆一次、删除一次，算法结束时所有 n 个元素均被正确合并。

时间复杂度：堆的初始建堆花费 $O(k)$ 。算法共执行 n 次 Extract-Min 操作和至多 n 次 Insert 操作，每次操作耗时 $O(\log k)$ （堆中元素数不超过 k ）。故总时间为：

$$O(k) + n \cdot O(\log k) + n \cdot O(\log k) = O(n \log k).$$

七. (10 分) 计算逆序对总数

(1) (5 分) 算法设计

算法思想：在归并排序的框架下统计逆序对。分治过程：

- **分解：**将数组 $A[1..n]$ 分为左右两半 $A[1..mid]$ 和 $A[mid+1..n]$ 。
- **递归：**递归计算左半部分和右半部分内部的逆序对数目。
- **合并：**在归并两个有序子数组的过程中，统计跨越左右两部分的逆序对。

Merge 步骤中统计逆序对：归并时，使用双指针 i （左半）和 j （右半）。当 $A[i] > A[j]$ 时，由于左半已排序， $A[i..mid]$ 中所有元素均大于 $A[j]$ ，因此产生 $mid - i + 1$ 个跨子数组的逆序对。将此值累加到计数器。

伪代码：

```

COUNT-INVERSIONS(A, left, right)
    if left >= right
        return 0
    mid = (left + right) / 2
    inv_left = COUNT-INVERSIONS(A, left, mid)
    inv_right = COUNT-INVERSIONS(A, mid+1, right)
    inv_cross = MERGE-AND-COUNT(A, left, mid, right)
    return inv_left + inv_right + inv_cross

MERGE-AND-COUNT(A, left, mid, right)
    i = left, j = mid+1, k = 0, count = 0
    temp[1..right-left+1]
    while i <= mid and j <= right
        if A[i] <= A[j]
            temp[++k] = A[i++]
        else
            temp[++k] = A[j++]
            count += mid - i + 1    // 关键：统计逆序对
    while i <= mid: temp[++k] = A[i++]
    while j <= right: temp[++k] = A[j++]
    copy temp back to A[left..right]
    return count

```

(2) (5 分) 正确性证明与时间复杂度分析

正确性：每个逆序对 (i, j) ($i < j$ 且 $A[i] > A[j]$) 有两种情况：

- i 和 j 在同一半内：由递归调用正确统计。
- i 在左半、 j 在右半：在 Merge 步骤中，当右半元素 $A[j]$ 被选中时，左半指针 i 指向第一个大于 $A[j]$ 的元素，由于左半有序， $A[i..mid]$ 均大于 $A[j]$ 且下标均小于 j ，恰好构成 $mid - i + 1$ 个逆序对。所有跨子数组逆序对被不重不漏地计数。

时间复杂度：设 $T(n)$ 为计算长度为 n 的数组逆序对的时间。递归式为：

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

其中 $\Theta(n)$ 为 Merge 步骤的时间。由主定理 ($a = b = 2$, $\log_b a = 1$, $f(n) = \Theta(n)$, 情形 2)，解得：

$$T(n) = \Theta(n \log n).$$

八. (10 分) 递归式 $T(n) = 3T(\frac{n}{3}) + n\sqrt{\log n}$ 的渐进上界

(1) (3 分) 尝试主方法

这里 $a = 3$, $b = 3$, 故 $n^{\log_b a} = n$ 。而 $f(n) = n\sqrt{\log n}$ 。

$f(n)$ 比 n 多一个 $\sqrt{\log n}$ 因子, 但不是多项式意义上的更大: 不存在常数 $\epsilon > 0$ 使 $n\sqrt{\log n} = \Omega(n^{1+\epsilon})$ (因为对任意 $\epsilon > 0$, $\frac{\sqrt{\log n}}{n^\epsilon} \rightarrow 0$)。因此不满足主定理第 3 种情形; 也不属于第 1、2 种情形, 主方法不能直接应用。

(2) (4 分) 递归树法

令 $L = \log_3 n$ (递归树的层数)。第 i 层 ($i = 0, 1, \dots, L-1$) 有 3^i 个规模为 $n/3^i$ 的子问题, 每个子问题的非递归代价为:

$$\frac{n}{3^i} \sqrt{\log\left(\frac{n}{3^i}\right)} = \frac{n}{3^i} \sqrt{\log_3 n \cdot \log 3 - i \cdot \log 3}$$

为简化, 令对数底为常数, 每个子问题代价简写为 $\frac{n}{3^i} \sqrt{L-i}$ (忽略常数因子)。第 i 层总代价为:

$$3^i \cdot \frac{n}{3^i} \sqrt{L-i} = n\sqrt{L-i}.$$

递归树共有 L 层。叶子层 (第 L 层) 有 $3^L = n$ 个规模为 1 的子问题, 叶子总代价为 $\Theta(n)$ 。

非叶子总代价为:

$$\sum_{i=0}^{L-1} n\sqrt{L-i} = n \sum_{t=1}^L \sqrt{t} = \Theta(n \cdot L^{3/2}) = \Theta(n(\log n)^{3/2}).$$

叶子项 $\Theta(n)$ 被上述项支配, 因此:

$$T(n) = \Theta(n(\log n)^{3/2}).$$

(3) (3 分) 代换法证明 (上界)

猜测: $T(n) \leq c \cdot n(\log n)^{3/2}$, 对充分大的 n 。

归纳假设: 设对所有 $m < n$, 有 $T(m) \leq c \cdot m(\log m)^{3/2}$ 。

代入递归式:

$$\begin{aligned} T(n) &= 3T\left(\frac{n}{3}\right) + n\sqrt{\log n} \\ &\leq 3 \cdot c \cdot \frac{n}{3} \left(\log \frac{n}{3}\right)^{3/2} + n\sqrt{\log n} \\ &= c \cdot n (\log n - \log 3)^{3/2} + n\sqrt{\log n}. \end{aligned}$$

目标：需证 $cn(\log n - \log 3)^{3/2} + n\sqrt{\log n} \leq cn(\log n)^{3/2}$ ，即：

$$n\sqrt{\log n} \leq cn[(\log n)^{3/2} - (\log n - \log 3)^{3/2}].$$

对 $f(x) = x^{3/2}$ 在区间 $[\log n - \log 3, \log n]$ 上应用拉格朗日中值定理：存在 $\xi \in (\log n - \log 3, \log n)$ ，使得

$$(\log n)^{3/2} - (\log n - \log 3)^{3/2} = f'(\xi) \cdot \log 3 = \frac{3}{2}\sqrt{\xi} \cdot \log 3.$$

由于 $\xi \geq \log n - \log 3 \geq \frac{1}{2} \log n$ （对充分大的 n ），有 $\sqrt{\xi} \geq \frac{1}{\sqrt{2}}\sqrt{\log n}$ 。因此：

$$(\log n)^{3/2} - (\log n - \log 3)^{3/2} \geq \frac{3 \log 3}{2\sqrt{2}} \sqrt{\log n}.$$

代入上式：

$$n\sqrt{\log n} \leq cn \cdot \frac{3 \log 3}{2\sqrt{2}} \sqrt{\log n} \implies c \geq \frac{2\sqrt{2}}{3 \log 3}.$$

选取 $c = \frac{2\sqrt{2}}{3 \log 3}$ （约为 1.18），归纳即成立。（注：还需处理基础情况，可选取充分大的常数覆盖。）

下界证明：类似可证 $T(n) \geq c' \cdot n(\log n)^{3/2}$ （猜测下界并用代换法）。

结论： $T(n) = \Theta(n(\log n)^{3/2})$ 。

九.（10 分）Randomize-In-Place 算法的均匀性

答案：不能产生均匀随机排列。

反例：取 $n = 3$ 。每次迭代独立地从 $\{1, 2, 3\}$ 中选择两个位置（可相同），共有 $3 \times 3 = 9$ 种等可能的有序选择。执行 3 次迭代，共有 $9^3 = 729$ 种等可能的随机选择序列。

若最终排列在 $3! = 6$ 个排列上均匀分布，则每个排列应由 $729/6 = 121.5$ 个序列产生——这不可能（因为 729 不能被 6 整除）。因此该算法对 $n = 3$ 不能产生均匀随机排列。

进一步验证：可计算恒等排列的概率。恒等排列要求经过 3 次交换后排列保持不变，其概率为 $5/27$ ，而均匀分布下每个排列的概率应为 $1/6 = 4.5/27$ ，二者不等。

本质原因：原始的 Randomize-In-Place 确保第 i 个位置的元素在剩余元素中均匀选择，从而产生 $n!$ 种等可能的执行路径。修改后的算法每次从全部 n 个位置中随机选择，破坏了这种一一对应关系，导致排列分布不均匀。

十.（15 分）线性时间选择算法（SELECT / 中位数的中位数算法）

(1)（5 分）算法步骤与伪代码

算法思想：将数组分成每组 5 个元素，找每组中位数，递归找中位数的中位数作为 pivot，保证划分的平衡性。

伪代码：

```
SELECT(A, left, right, i)
// 返回A[left..right]中第i小的元素
1.  n = right - left + 1
2.  if n <= 5
3.      直接排序A[left..right], 返回第i小的元素
4.  将A[left..right]分成 n/5 组, 每组5个元素 (最后一组可能不足5个)
5.  对每组排序, 找出中位数
6.  将所有中位数置于数组B中
7.  pivot = SELECT(B, 1, n/5, n/5/2) // 递归找中位数的中位数
8.  以pivot划分A[left..right] (同快速排序的PARTITION)
9.  设pivot在划分后的位置为p, 其在子数组中的相对排名为k = p - left + 1
10. if i == k
11.     return A[p]
12. elseif i < k
13.     return SELECT(A, left, p-1, i)
14. else
15.     return SELECT(A, p+1, right, i-k)
```

(2) (5 分) 时间复杂度递归式

设 $T(n)$ 为对 n 个元素运行 SELECT 的最坏情况时间。

- 步骤 5-6: 对 $\lceil n/5 \rceil$ 组各自排序找中位数, 每组至多 5 个元素, 每组排序耗时 $O(1)$, 总计 $O(n)$ 。
- 步骤 7: 递归找中位数的中位数, 问题规模为 $\lceil n/5 \rceil$, 耗时 $T(\lceil n/5 \rceil)$ 。
- 步骤 8: 划分操作耗时 $O(n)$ 。
- 步骤 12 或 14: 递归处理一侧子数组。可以证明, 至少有一半的中位数 (即至少 $\lceil n/10 \rceil - 1$ 个中位数) 大于等于 pivot (类似地, 也至少有一半小于等于 pivot), 故 pivot 至少比 $\lceil n/5 \rceil / 2 - 1 \approx n/10$ 个中位数大, 而每个中位数又至少比 2 个元素大 (在 5 个元素组中), 因此 pivot 至少比 $3n/10 - 6$ 个元素大。同理, pivot 也至少比 $3n/10 - 6$ 个元素小。因此递归的子问题规模至多为 $n - (3n/10 - 6) \approx 7n/10 + 6$ 。

故递归式为:

$$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{10} + 6\right) + O(n).$$

(3) (5 分) 将每组大小从 5 改为 3 的分析

递归式的变化:

若每组 3 个元素, 则:

- 中位数个数为 $\lceil n/3 \rceil$ 。
- 递归找中位数的中位数耗时 $T(\lceil n/3 \rceil)$ 。
- 每个中位数来自 3 个元素组，故至少比组内另外 1 个元素大（中位数大于组内约一半元素）。至少有一半中位数（即 $\lceil n/6 \rceil$ ）大于等于 pivot。每个这样的中位数代表组内至少 2 个元素 pivot（中位数本身 + 比它小的那个）。因此 pivot 至少比 $2 \times \lceil n/6 \rceil \approx n/3$ 个元素大。
- 同理，pivot 也至少比约 $n/3$ 个元素小。
- 递归子问题规模至多为 $n - n/3 = 2n/3$ 。

此时递归式为：

$$T(n) \leq T\left(\left\lceil \frac{n}{3} \right\rceil\right) + T\left(\frac{2n}{3} + O(1)\right) + O(n).$$

能否保证 $O(n)$ ？

用代换法验证。猜测 $T(n) \leq cn$ 。

$$\begin{aligned} T(n) &\leq c \cdot \frac{n}{3} + c \cdot \frac{2n}{3} + an \quad (a \text{ 为 } O(n) \text{ 的常数系数}) \\ &= c \cdot \frac{n}{3} + c \cdot \frac{2n}{3} + an \\ &= cn + an > (c + a)n. \end{aligned}$$

无法满足 $T(n) \leq cn$ ，因为 $\frac{1}{3} + \frac{2}{3} = 1$ ，加上线性额外开销后总和超过 cn 。

更严格的分析：考虑递归树法。设递归式 $T(n) = T(n/3) + T(2n/3) + O(n)$ 。

该递归式与随机化快速排序的最坏情况递归式类似。其解为 $T(n) = \Theta(n \log n)$ ，而非 $O(n)$ 。事实上，递归树在第 k 层的最大子问题规模为 $n \cdot (2/3)^k$ ，递归深度为 $\Theta(\log n)$ 而非 $O(1)$ 。每层总代价为 $O(n)$ （实际上不同分支代价可叠加），总代价为 $O(n \log n)$ 。

结论：当每组大小从 5 改为 3 时，算法不能保证最坏情况 $O(n)$ 的时间复杂度，此时时间复杂度退化为 $\Theta(n \log n)$ 。这是因为 $\frac{1}{3} + \frac{2}{3} = 1$ ，递归式中子问题的系数之和等于 1，导致递归树深度变为对数级别。而当每组大小为 5 时，系数之和 $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ ，保证了递归树每层规模几何级数递减，从而总时间为 $O(n)$ 。

十一. (10 分) 全域哈希与碰撞概率

(1) (3 分) 全域哈希族定义

设关键字全集为 U , 哈希表槽数为 m 。 $\mathcal{H}$ 是一组从 U 到 $\{0, 1, \dots, m-1\}$ 的哈希函数。若对任意两个不同关键字 $x \neq y$, 从 $\mathcal{H}$ 中均匀随机选取 h 时都有:

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m},$$

则称 $\mathcal{H}$ 为全域哈希族 (Universal Hash Family)。

(2) (5 分) 碰撞概率证明

题中哈希函数可写为 $h_a(x) = \lfloor m\{ax\} \rfloor$, 其中 $\{u\} = u - \lfloor u \rfloor$ 表示小数部分, a 从 $(0, 1)$ 上均匀随机选取。

令 $d = |x_1 - x_2| \geq 1$ (d 为正整数)。若 $h_a(x_1) = h_a(x_2)$, 则 $\{ax_1\}$ 与 $\{ax_2\}$ 落入同一个长度为 $1/m$ 的区间 $[t/m, (t+1)/m)$, 因此它们在单位圆周上的距离小于 $1/m$, 从而:

$$\{a(x_1 - x_2)\} = \{ad\} \in [0, 1/m) \cup (1 - 1/m, 1).$$

因为 d 是正整数且 a 在 $(0, 1)$ 上均匀随机选取, 所以 $\{ad\}$ 在 $[0, 1)$ 上均匀分布 (小数部分均匀分布)。因此:

$$\Pr[h_a(x_1) = h_a(x_2)] \leq \Pr[\{ad\} \in [0, 1/m) \cup (1 - 1/m, 1)] \leq \frac{2}{m}.$$

(3) (2 分) 不成功查找的期望时间

对一个不在表中的关键字 x , 链中期望碰撞元素数至多为:

$$\sum_{y \text{ 在表中}} \Pr[h(x) = h(y)] \leq n \cdot \frac{2}{m} = 2\alpha.$$

其中 $\alpha = n/m$ 为装载因子。因此, 一次不成功查找的期望时间上界为 $\Theta(1 + 2\alpha) = \Theta(1 + \alpha)$ 。

对比: 在标准全域哈希 (碰撞概率 $\leq 1/m$) 下, 不成功查找的期望时间为 $\Theta(1 + \alpha)$ 。本题中的哈希族碰撞概率上界放大了常数因子 2, 但不影响渐进界, 期望时间仍为 $\Theta(1 + \alpha)$ 。

十二. (10 分) 随机化快速排序的期望比较次数与方差

(1) (6 分) 求 $\Pr[X_{ij} = 1]$ 与期望比较次数

概率推导:

在随机化快速排序中，两个元素 z_i 与 z_j ($i < j$) 会被直接比较，当且仅当集合 $\{z_i, z_{i+1}, \dots, z_j\}$ 中第一个被选为 pivot 的元素是 z_i 或 z_j 。若第一个 pivot 是中间某个 z_k ($i < k < j$)，则 z_i 与 z_j 会被分到不同子数组，之后不再比较。

该区间共有 $j - i + 1$ 个元素，每个元素等概率率先成为 pivot，因此：

$$\Pr[X_{ij} = 1] = \frac{2}{j - i + 1}.$$

期望比较次数：

令总比较次数 $X = \sum_{1 \leq i < j \leq n} X_{ij}$ 。由期望的线性性：

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}.$$

令 $k = j - i$ ，则 $k = 1, 2, \dots, n - 1$ ，且固定 k 时共有 $n - k$ 对满足 $j - i = k$ 的元素对。因此：

$$\mathbb{E}[X] = \sum_{k=1}^{n-1} (n - k) \frac{2}{k + 1} \leq \sum_{k=1}^{n-1} n \cdot \frac{2}{k + 1} = 2n \sum_{k=1}^{n-1} \frac{1}{k + 1}.$$

由于 $\sum_{k=1}^{n-1} \frac{1}{k+1} \leq \sum_{k=1}^n \frac{1}{k} = H_n = O(\log n)$ （调和级数），所以：

$$\mathbb{E}[X] \leq 2nH_n = O(n \log n).$$

即随机化快速排序的期望比较次数为 $O(n \log n)$ 。

(2) (4 分) **证明** $\text{Var}[X] = O(n^2)$

思路：利用方差展开式和协方差分析，证明比较次数集中在期望值附近。

优先级模型：

另一种理解随机化快速排序的方式是：每个元素 z_t 预先获得一个独立随机优先级 $U_t \sim \text{Uniform}(0, 1)$ （连续分布，无平局）。在每个子数组中，优先级最小的元素被选为 pivot。等价地， $X_{ij} = 1$ 当且仅当区间 $[i, j]$ 中优先级最小的元素出现在端点 i 或 j 。

方差分解：

$$\text{Var}[X] = \sum_{1 \leq i < j \leq n} \text{Var}[X_{ij}] + \sum_{(i,j) \neq (k,l)} \text{Cov}(X_{ij}, X_{kl}).$$

(a) 单项方差之和：

由于 X_{ij} 为指示器随机变量，

$$\text{Var}[X_{ij}] = \Pr[X_{ij} = 1] \cdot (1 - \Pr[X_{ij} = 1]) \leq \Pr[X_{ij} = 1] = \frac{2}{j - i + 1}.$$

故：

$$\sum_{i < j} \text{Var}[X_{ij}] \leq \sum_{i < j} \frac{2}{j - i + 1} = \mathbb{E}[X] = O(n \log n) = O(n^2).$$

(b) 协方差之和：

考虑任意两对不同的指标对 (i, j) 和 (k, l) ($i < j, k < l$)。

情形 1： 区间 $[i, j]$ 与 $[k, l]$ 不相交 ($j < k$ 或 $l < i$)。此时两个事件依赖的元素集合互不相交，由优先级独立性知 X_{ij} 与 X_{kl} 独立，故：

$$\text{Cov}(X_{ij}, X_{kl}) = 0.$$

情形 2： 区间相交。我们断言 $\text{Cov}(X_{ij}, X_{kl}) \leq 0$ ，即 X_{ij} 与 X_{kl} 呈负相关。

负相关性的直观解释： 若 $X_{ij} = 1$ 发生，意味着 z_i 或 z_j 是 $[i, j]$ 中第一个被选为 pivot 的元素。无论哪种情况，该 pivot 将数组划分后，凡是跨越该 pivot 的区间对（即一个在 pivot 左侧，另一个在 pivot 右侧）就不可能再被比较。特别地，任何与 $[i, j]$ 相交但未完全落入同一侧的区间 $[k, l]$ ，其中元素被比较的概率将降低。因此：

$$\Pr[X_{kl} = 1 \mid X_{ij} = 1] \leq \Pr[X_{kl} = 1],$$

即：

$$\mathbb{E}[X_{ij} X_{kl}] \leq \mathbb{E}[X_{ij}] \cdot \mathbb{E}[X_{kl}],$$

从而 $\text{Cov}(X_{ij}, X_{kl}) \leq 0$ 。

更严格的分析（分情况讨论）：

设四个指标 $i < j, k < l$ ，区间相交。考虑以下子情况：

- **一区间包含另一区间：** 不妨设 $i \leq k < l \leq j$ 。若 $X_{ij} = 1$ ，优先序最小的元素在 z_i 或 z_j 。若该元素不在 $[k, l]$ 内（即 pivot 是 z_i 且 $i < k$ ，或 pivot 是 z_j 且 $l < j$ ），则 z_k 与 z_l 落入同一侧，仍可能后续被比较。但总体而言， $X_{ij} = 1$ 的条件增加了 z_k 和 z_l 被分割的概率，因此 $\Pr[X_{kl} = 1 \mid X_{ij} = 1] \leq \frac{2}{l - k + 1} = \Pr[X_{kl} = 1]$ 。
- **区间交错：** $i < k < j < l$ 。此时若 $X_{ij} = 1$ 且 pivot 为 z_j ，则 z_j 的划分使得 z_k （在 z_j 左侧）与 z_l （在 z_j 右侧）分入不同子数组， $X_{kl} = 0$ 必然为 0。若 pivot 为 z_i ($z_i < z_k$)，则 z_k 与 z_l 仍在同一侧， X_{kl} 仍可能为 1。综合两种可能，条件概率仍不超过无条件概率。事实上， $X_{ij} = 1$ 时 $X_{kl} = 1$ 至少排除了「pivot 为 z_j 」这一情况，在该情况下 X_{kl} 必定为 0。

以上分析表明，对所有相交区间对，均有 $\text{Cov}(X_{ij}, X_{kl}) \leq 0$ 。

(c) 综合：

将负相关（或独立）性质代入方差公式：

$$\text{Var}[X] = \underbrace{\sum_{i < j} \text{Var}[X_{ij}]}_{\leq \mathbb{E}[X] = O(n \log n)} + \underbrace{\sum_{\text{区间相交}} \text{Cov}(X_{ij}, X_{kl})}_{\leq 0} + \underbrace{\sum_{\text{区间不相交}} 0}_{= 0}.$$

因此：

$$\text{Var}[X] \leq \mathbb{E}[X] = O(n \log n) = O(n^2).$$

结论： $\text{Var}[X] = O(n^2)$ 。由切比雪夫不等式（Chebyshev's Inequality）：

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{\text{Var}[X]}{t^2}.$$

取 $t = \omega(1) \cdot n$ （例如 $t = n \log n$ ），可知 $|X - \mathbb{E}[X]|$ 以高概率为 $O(n)$ 量级，因此比较次数确实集中在期望值 $\mathbb{E}[X] = \Theta(n \log n)$ 附近，标准差 $O(n)$ 远小于均值 $\Theta(n \log n)$ 。

算法设计与分析期末考试试卷

Amadeus

一、选择题

第一题（拓扑排序与 DAG）

以下关于拓扑排序的说法，正确的是：

- A. 任何有向图都存在至少一个拓扑排序
- B. 一个有向无环图（DAG）的拓扑排序是唯一的
- C. 若一个有向图包含环，则不存在该图的拓扑排序
- D. 拓扑排序可以基于广度优先搜索（BFS）实现，无法基于深度优先搜索（DFS）实现
- E. 拓扑排序算法的时间复杂度为 $O(n \log n)$

第二题（NP 完全性（NPC）的定义——实际考的比这个题简单）

设 A 、 B 为两个判定问题。若对任意问题实例 I ，存在多项式时间可计算的函数 f ，使得 $I \in A$ 当且仅当 $f(I) \in B$ ，则称 A 可多项式归约到 B ，记为 $A \leq_p B$ 。

根据 NPC 理论，以下哪项是判定问题 B 属于 NPC 类的充要条件？

- A. B 属于 NP，且存在某个 NP 问题可多项式归约到 B
- B. B 属于 NP，且所有 NP 问题均可多项式归约到 B
- C. B 属于 P，且 SAT 问题可多项式归约到 B
- D. B 属于 NP-Hard，且 B 可在指数时间内求解
- E. B 可在多项式时间内验证解的正确性

第三题（DFS 边的分类——实际考的比这个题简单）

对无向连通图进行深度优先搜索（DFS）时，每条边被分类为树边或回边（后向边）。而对有向图进行 DFS 时，边可分为四种类型：树边（Tree Edge）、前向边（Forward Edge）、后向边（Back Edge）和横向边（Cross Edge）。

以下关于 DFS 边分类的说法中，错误的是：

- A. 在无向图的 DFS 中，不可能出现横向边
- B. 在有向图 DFS 中，若边 (u, v) 满足发现时间 $d[u] < d[v] < f[v] < f[u]$ 且 v 不是 u 在 DFS 树中的直接子节点，则 (u, v) 是前向边
- C. 在有向图 DFS 中，后向边的存在等价于图中存在环
- D. 在有向图 DFS 中，横向边总是从较晚发现的结点指向较早发现的结点
- E. 在无向图的 DFS 中，非树边一定连接祖先和后代结点

第四题

我忘了... 应该是简单题

第五题（多选题）

考察了一些时间复杂度，需要注意 O 和 Θ ，建议多看几遍笔记和板书

第六题（half-pop 操作的摊还分析）

某数据结构支持三种操作：PUSH（压入一个元素）、POP（弹出栈顶元素）以及 HALF-POP（弹出当前栈中约一半的元素）。HALF-POP 的具体语义为：若当前栈中有 k 个元素，则该操作弹出 $\lfloor k/2 \rfloor$ 个元素（即弹出栈顶的一半，保留另一半在栈中），并返回被弹出的这些元素。

要求：使用聚合分析（核方法，Accounting Method）和势能法（Potential Method）两种方法，分别证明：对任意由 n 个 PUSH、POP 和 HALF-POP 操作构成的序列，总摊还时间复杂度为 $O(n)$ 。

- (1) 使用核方法（记账法）：为每种操作设定摊还代价，并说明在操作序列的任何时刻信用（credit）非负，从而总摊还代价为实际代价的上界。
- (2) 使用势能法：定义合适的势函数 Φ ，计算每种操作的摊还代价 $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ ，并证明总摊还代价为 $O(n)$ 。

第七题（House Robber 问题）

你是一名专业的小偷，计划沿街盗窃房屋。街道上的房屋排成一条直线（单排排列，首尾不相连）。每间房屋内都藏有一定数额的现金（用非负整数表示）。唯一的限制条件是：这条街上的房屋装有联网的防盗系统，如果相邻的两间房屋在同一晚上被闯入，系统会自动报警。

给定一个整数数组 $nums[0..n-1]$ ，其中 $nums[i]$ 代表第 i 间房屋内的现金金额。你需要计算出：在不触发警报的前提下，一夜之内能够盗窃到的**最高总金额**。

- (1) 给出状态定义与状态转移方程（递推式），并说明最优子结构性质。
- (2) 证明你的递推式的正确性（即为什么该递推式给出了最优解）。
- (3) 写出使用 $O(1)$ 额外空间的算法实现（伪代码或详细描述），并分析时间复杂度。说明如何将空间复杂度从 $O(n)$ 优化到 $O(1)$ 。

第八题（带截止时间和收益的任务调度）

给定 n 个任务，每个任务 i 有一个截止时间（deadline） d_i 和一个收益（profit） $p_i > 0$ 。每个任务恰好占用 1 个单位时间，所有任务的截止时间满足 $1 \leq d_i \leq n$ 。共有 n 个时间单位（时隙 $1, 2, \dots, n$ ）可用于执行任务。每个时隙最多安排一个任务，每个任务至多被安排一次。若任务 i 在其截止时间 d_i 或之前完成，则获得收益 p_i ；否则收益为 0。目标是选择并安排一组任务，使得总收益最大化。

- (1) 给出贪心规则（即每次选择哪个任务、安排在什么时间），设计一个完整的贪心算法，并写出伪代码或详细的算法描述。
- (2) 证明该贪心算法的正确性（使用交换论证或拟阵理论）。
- (3) 分析算法的时间复杂度，并说明使用什么数据结构可以进行优化。

第九题（安全边定理与 Kruskal 算法）

最小生成树（MST）问题：给定连通无向带权图 $G = (V, E)$ ，权函数 $w : E \rightarrow \mathbb{R}$ ，求一棵边权之和最小的生成树。

- (1)（4 分）叙述「安全边定理」（Safe Edge Theorem）。
- (2)（4 分）给出安全边定理的完整证明。
- (3)（4 分）利用安全边定理，证明 Kruskal 算法的正确性。

第十题（13 分）Dijkstra 算法的变种

给定带权有向图 $G = (V, E)$ ，其中所有边的权重满足 $w(u, v) > 0$ 。设 $s, t \in V$ 分别为源点和汇点。

- (1)（4 分）简述 Dijkstra 最短路径算法的核心思想、关键步骤和正确性前提。说明为何算法要求所有边权非负。

(2) (9 分) 现在考虑以下变种问题：在图中**恰好选择一条边**，将其权重变为原来的一半（即 $w'(e) = w(e)/2$ ），其余边权重不变。设计一个算法，在所有可能的「选择一条边并将其权重减半」的方案中，找出能使 s 到 t 的最短路径长度最小化的方案。要求：

- 描述算法思想并给出关键步骤。
- 分析算法的时间复杂度。
- 证明算法的正确性（说明为何该算法确实找到了最优方案）。

第十一题 (15 分) 多重背包问题与 FPTAS

设有 n 种物品，第 i 种物品有价值 $v_i > 0$ 、重量 $w_i > 0$ 以及数量 c_i （表示第 i 种物品共有 c_i 件可用）。背包容量为 W 。目标是选择每种物品的若干件（不超过其数量限制），使得装入背包的总重量不超过 W ，且总价值最大化。

- (1) (5 分) 使用**二进制拆分**方法，证明多重背包问题在多项式时间内等价于 0-1 背包问题。
- (2) (5 分) 基于 0-1 背包的 FPTAS（完全多项式时间近似方案）思路，为上述多重背包问题设计一个近似算法，使得对于任意给定的 $\varepsilon \in (0, 1)$ ，算法输出的解的价值至少为最优解的 $(1 - \varepsilon)$ 倍。描述算法步骤。
- (3) (5 分) 证明该近似算法的正确性（近似比为 $1 - \varepsilon$ ），并给出算法的时间复杂度（用 n 、 W 、 $\max c_i$ 和 $1/\varepsilon$ 表示）。